Implementing a Generalized N-gram Language Model

NLP Laboratory Experiment • Execution Guide

1. Why the earlier program was not checking N-grams

The earlier code used only the current word and the following word. Therefore, it implemented a **Bigram model** only. A generalized N-gram model must allow the user to choose N and form sequences of N consecutive words.

2. Understanding Unigram, Bigram, Trigram and 4-gram

Model	N	Example from “the cat is sitting on the mat”
Unigram	1	the | cat | is | sitting | on | the | mat
Bigram	2	the cat | cat is | is sitting | sitting on | ...
Trigram	3	the cat is | cat is sitting | is sitting on | ...
4-gram	4	the cat is sitting | cat is sitting on | ...

For an N-gram model, the preceding context contains N−1 words, and the final word in the sequence is the word being predicted.

3. Aim

To implement a generalized N-gram language model that predicts the next word using the preceding sequence of words, and to test different values of N.

4. Software requirements

• Python 3.x
• Visual Studio Code or Python IDLE
• No external Python libraries are required.

5. Complete Python program

```
from collections import defaultdict, Counter

# STEP 1: Training data
text = """
the cat is sitting on the mat
the cat is playing with a ball
the dog is playing in the garden
the dog is sitting near the door
the cat is sleeping on the sofa
"""

# STEP 2: Tokenization
words = text.lower().split()

# STEP 3: Get N value from user
n = int(input("Enter N value (1, 2, 3 or 4): "))

if n < 1:
    raise ValueError("N must be 1 or greater.")

# STEP 4: Create N-gram model
ngram_model = defaultdict(Counter)

for i in range(len(words) - n + 1):
    ngram = tuple(words[i:i+n])
```

```
    # Previous n-1 words form the context
    context = ngram[:-1]

    # Last word is the predicted word
    next_word = ngram[-1]

    ngram_model[context][next_word] += 1

# STEP 5: Display generated N-grams
print("\nGenerated N-grams:")
for context, predictions in ngram_model.items():
    for word, count in predictions.items():
        print(context, "->", word, ":", count)

# STEP 6: Get input sequence
user_input = input("\nEnter preceding words: ").lower().split()

# STEP 7: Extract context
context = tuple(user_input[-(n-1):]) if n > 1 else ()

# STEP 8: Predict next word
print("\nPrediction:")
if context in ngram_model:
    prediction = ngram_model[context].most_common(1)
    print("Input context:", context)
    print("Predicted next word:", prediction[0][0])
else:
    print("No matching N-gram found.")
```

6. How to execute the program

Step 1 – Open an editor: Open Visual Studio Code or Python IDLE.

Step 2 – Create a file: Create a new Python file and save it as **ngram_model.py**.

Step 3 – Enter the code: Copy the complete program above into the file and save it.

Step 4 – Open the terminal: In VS Code, select Terminal → New Terminal.

Step 5 – Run the program: Type `python ngram_model.py` and press Enter. If your system uses the Python launcher, try `py ngram_model.py`.

Step 6 – Enter N: Enter 2 for Bigram, 3 for Trigram, or 4 for a 4-gram.

Step 7 – Enter the context: Enter the preceding N–1 words, then press Enter to see the predicted next word.

7. Sample experiments and expected predictions

Experiment	N value	Input context	Expected prediction
Bigram	2	cat	is
Trigram	3	the cat	is
4-gram	4	the cat is	sitting

Note: These are expected results for the supplied training sentences. If a context does not occur in the training data, the program reports that no matching N-gram was found.

8. Explanation of the important code

Creating an N-gram

```
ngram = tuple(words[i:i+n])
```

This extracts N consecutive words. For N = 3, an example sequence is `('the', 'cat', 'is')`.

Separating context and predicted word

```
context = ngram[:-1]\nnext_word = ngram[-1]
```

For the trigram ('the', 'cat', 'is'), the context is ('the', 'cat') and the next word is 'is'. The Counter records how often each next word follows a context. `most_common(1)` selects the most frequent continuation.

9. Observation table

Model	N	Number of preceding words
Unigram	1	0
Bigram	2	1
Trigram	3	2
4-gram	4	3

Unigram is a frequency-based word model and does not use preceding words as context. For N ≥ 2, the model uses the previous N–1 words.

10. Viva-voce questions

1. What is an N-gram language model?
2. What is the difference between Unigram, Bigram and Trigram?
3. What is next-word prediction?
4. What is the role of training data?
5. Why does a Trigram model use two preceding words?
6. What happens when an input context is not found in the training data?
7. What is the limitation of using a large N value?

11. Result

A generalized N-gram language model was implemented in Python. The program accepts different values of N, generates the corresponding N-gram counts, and predicts the most frequent next word for a supplied context.

Teaching note

Increasing N allows the model to use a longer context, but larger N-grams are more likely to be unseen in a small training corpus. This is one reason practical language models use techniques such as smoothing and backoff. The supplied program is a simple educational implementation.